Unit testing in Python

Unittest

```
import unittest
def square(n):
   return n*n

def cube(n):
   return n*n*n

class Test(unittest.TestCase):
   def test_square(self):
      self.assertEquals(square(2), 4)

   def test_cube(self):
      self.assertEquals(cube('abc'), 8)
```

UnitTest Framework - Skip Test

It is possible to skip individual test method or TestCase class, conditionally as well as unconditionally. The framework allows a certain test to be marked as an 'expected failure'. This test will 'fail' but will not be counted as failed in TestResult.

```
import unittest
   def add(x,y):
      return x+y
class SimpleTest(unittest.TestCase):
   @unittest.skip("demonstrating skipping")
   def testadd1(self):
      self.assertEquals(add(4,5),9)
```

S.No.	Method & Description
1	**unittest.skip(reason)** Unconditionally skip the decorated test. *reason* should describe why the test is being skipped.
2	**unittest.skipIf(condition, reason)** Skip the decorated test if condition is true.
3	**unittest.skipUnless(condition, reason)** Skip the decorated test unless condition is true.
4	**unittest.expectedFailure()** Mark the test as an expected failure. If the test fails when run, the test is not counted as a failure.

```
import unittest
class suiteTest(unittest.TestCase):
  a = 50
  b = 40

  def testadd(self):
    result = self.a+self.b
    self.assertEqual(result,100)
  @unittest.skipIf(a>b, "Skip over this routine")
  def testsub(self):
    result = self.a-self.b
    self.assertTrue(result == -10)
  @unittest.skipUnless(b == 0, "Skip over this routine")
  def testdiv(self):
    result = self.a/self.b
    self.assertTrue(result == 1)
  @unittest.expectedFailure
  def testmul(self):
    result = self.a*self.b
    self.assertEqual(result == 0)
```

```
import unittest
class TestSkippingclass(unittest.TestCase):
  var = int(input())
@unittest.skip("example of skipping")
def test_example(self):
  self.var1 = 1
@unittest.skipIf(var < 0, "number is less than 0")
def test_example1(self):
  print("PASSED")
@unittest.skipUnless(var == 0, "number is greater than zero")
def test_example2(self):
  print("PASSED")
```

Test suite vs test cases vs text runner vs text fixture

A **test case** is a class that represents an individual unit of testing. That's the place where you make sure your code works correctly. It contains fixtures and calls to assert methods to check for and report failures.

A **test suite** is just a bunch of test cases together.

A **test runner** is a script that takes care of running the test suite.

Test fixtures provide the necessary initial conditions and configure a fixed state for software tests. They also clean up after testing completes. Test fixtures provide a stable baseline for consistent, repeatable testing and allow for test initialization and clean up code to remain separate from the tests themselves.

Test suite

```
#import usertest
#import configtest # first test
import unittest   # second test

class ConfigTestCase(unittest.TestCase):
   def setUp(self):
      print 'stp'
      ##set up code

   def runTest(self):
      #runs test
      print 'stp'


mySuit= unittest.TestSuite()
mySuit.addTest(unittest.makeSuite(ConfigTestCase))

runner=unittest.TextTestRunner()
runner.run(mySuit)
```

Test fixtures

It happens that **in specific scenarios**, unit tests as we write them **might become pure overhead**. How come? Because we repeat the same structures, and we write the same boilerplate code again and again, without really thinking about how these particular tests could become more pleasant to maintain and scale. Why should I care about looking for alternatives? Because it can save you time, it **can document your codebase through well organized and self-explanatory test fixtures**.
Sometimes the same test fixture can apply to several tests. For example, each test in a test suite used to exercise an interface to a database may need to configure (or mock) a database connection. It can be time consuming or resource intensive to bring up and tear down this connection for each test. Thankfully, the unittest framework provides three scopes for test fixtures; fixtures can be defined at the individual test, class or module level.
Test fixtures are methods and functions that run before and after a test.

- The intent is to provide developers hooks to set up preconditions needed for the test, and cleanup after the test.
- In many cases, this will be allocating, opening, or connecting to some resource in the setUp, and deallocating, closing, or disconnecting in the tearDown.

The most common fixture methods are setUp and tearDown.
The setUp() method runs before every test.
The tearDown() method runs after every test.
The setUp() and tearDown() methods allow you to define instructions that will be executed before and after each test method. If setUp() succeeded, tearDown() will be run whether the test method succeeded or not.
What is the difference between setUp() and setUpClass() in the Python unittest framework?

The main difference (as noted in the answer by Benjamin Hodgson) is that setUpClass is called only once and that is before all the tests, while setUp is called immediately before each and every test.
setUpClass()

A class method called before tests in an individual class are run. setUpClass is called with the class as the only argument and must be decorated as a classmethod():

@classmethod
def setUpClass(cls):
    ...

The setUpClass method is for expensive elements that you would rather only have to do once, such as opening a database connection, opening a temporary file on the filesystem, loading a shared library for testing, etc. Doing such things before each test would slow down the test suite too much, so we just do it once before all the tests

```
class Example(unittest.TestCase):
  @classmethod
def setUpClass(cls):
  print("setUpClass")

def setUp(self):
  print("setUp")

def test1(self):
  print("test1")

def test2(self):
  print("test2")

def tearDown(self):
  print("tearDown")

@classmethod
def tearDownClass(cls):
  print("tearDownClass")
When you run this test, it prints:
setUpClass
setUp
test1
tearDown
setUp
test2
tearDown
tearDownClass
```

Class and module level fixtures are implemented in TestSuite. When the test suite encounters a test from a new class then tearDownClass() from the previous class (if there is one) is called, followed by setUpClass() from the new class.
Similarly if a test is from a different module from the previous test then `tearDownModule` from the previous module is run, followed by `setUpModule` from the new module
After all the tests have run the final `tearDownClass` and `tearDownModule` are run.
A class method called before tests in an individual class are run. setUpClass is called with the class as the only argument and must be decorated as a classmethod():

```
@classmethod
def setUpClass(cls):
   ...
```

def setUpModule():

createConnection()

…..

def tearDownModule():

closeConnection()

setUp () and tearDown() methods : Sometimes the test code can be repetitive and hence we would end up writing the same code again and again. In this case we have a method called SetUp() method. For example let us consider the below code:

```
import unittest
class TestRepetitiveCode(unittest.TestCase):
 def test_absolute(self):
 self.var = -10
self.assertEqual(self.var, -10)

def test_overwrite(self):
 self.var = -10
self.assertTrue(self.var < 0)
```

In this case the self.var is declared in both the methods. If same variable or same piece of code is used in multiple methods then it becomes a tiring process to re-type the code again and again.
In this case we use the SetUp() method shown as below:

```
import unittest
class TestRepetitiveCode(unittest.TestCase):
 def setUp(self):
 self.var = -10
def test_absolute(self):
```

self.assertEqual(self.var, -10)
def test_overwrite(self):
self.assertTrue(self.var < 0)

```
import unittest

def fib(n):
    return 1 if n<=2 else fib(n-1)+fib(n-2)

def setUpModule():
        print("setup module")
def tearDownModule():
        print("teardown module")

class TestFib(unittest.TestCase):

    def setUp(self):
        print("setUp")
        self.n = 10
    def tearDown(self):
        print("tearDown")
        del self.n
    @classmethod
    def setUpClass(cls):
        print("setUpClass")
    @classmethod
    def tearDownClass(cls):
        print("tearDownClass")
    def test_fib_assert_equal(self):
        self.assertEqual(fib(self.n), 55)
    def test_fib_assert_true(self):
        self.assertTrue(fib(self.n) == 55)

if __name__ == "__main__":
    unittest.main()
```

output:

```
$ python test.py
setup module
setUpClass
setUp
tearDown
.setUp
tearDown
```

```
.tearDownClass
teardown module
```

To summarize:

1. setUp() and tearDown() methods are automatically used when they are available in your classes inheriting from unittest.TestCase.
2. They should be named setUp() and tearDown(), only those are used when test methods are executed.

To establish fixtures for each individual test case, override setUp() on the TestCase. To clean them up, override tearDown(). To manage one set of fixtures for all instances of a test class, override the class methods setUpClass() and tearDownClass() for the TestCase. And to handle especially expensive setup operations for all of the tests within a module, use the module-level functions setUpModule() and tearDownModule().

Pytest

Pytest in python is another framework that is used for testing. With pytest, we can run several tests in parallel. Because of that a considerable amount of computation time is saved. Using pytest is simple and does not involve complex codes.

```
def test_square():
  n = 2
  assert n*n == 4

def test_cube():
  n = 2
  assert n*n*n == 8
```

Python Unittest vs Pytest

Both unittest and pytest are testing frameworks in python. Unittest is the testing framework set in python by default. In unittest, we create classes that are derived from the *unittest.TestCase* module. It comes in handy because it is universally understood.

Whereas using pytest involves a compact piece of code. Pytest has rich inbuilt features which require less piece of code compared to unittest. In the case of unittest, we have to import a module, create a class and then define testing functions inside the class. But in the case of pytest, we have to define the functions and assert the conditions inside them.

```
import pytest
def test_file1_method1():
        x=5
        y=6
        assert x+1 == y,"test failed"
        assert x == y,"test failed"
def test_file1_method2():
        x=5
        y=6
```

```
assert x+1 == y,"test failed"
```

unittest follows follows OO design. You design a class , inherit it,, the methods define test cases (as functions).
In contrast to it, pytest is available that is extensive and easy to use. It does not necessitates OO design.
Continuing with the same example of phonebook.

```
from phonebook import *

def test_search():
phonebook=Phonebook()
phonebook.add("Ali","01234")
assert "01234" == phonebook.search("Ali")
```

it is not available by default..like unittest. You need to install it first!

How To Use Pytest Using Python

- Create a Python file with the name `mathlib.py`.
- Add the basic Python functions to it as below.

Example 1:

Create a file with the name "algo.py" and insert the below.

```
import pytest

def area_of_rectangle(width, height):
area = width*height
return area

def perimeter_of_rectangle(width, height):
perimeter = 2 * (width + height)
return perimeter
```

Create a file with the name "test_algo.py" in the same directory.

```
import algo

def test_area():
output = algo.area_of_rectangle(2,5)
assert output == 10

def test_perimeter():
output = algo.perimeter_of_rectangle(2,5)
assert output == 14
```

```

To run….Type: pytest -v test_algo.py

**Markers**

You can “mark” a test function with custom metadata like this:

```
content of test_server.py

import pytest

@pytest.mark.webtest
def test_send_http():
    pass # perform some webtest test for your app

def test_something_quick():
    pass

def test_another():
    pass

class TestClass:
    def test_method(self):
        pass
```

You can then restrict a test run to only run tests marked with webtest:

```
$ pytest -v -m webtest
```

Or the inverse, running all tests except the webtest ones:

```
$ pytest -v -m "not webtest"
```

A function is marked as a fixture by −

@pytest.fixture

```
import pytest

@pytest.fixture
def input_value():
   input = 39
   return input

def test_divisible_by_3(input_value):
   assert input_value % 3 == 0
```
```

```
def test_divisible_by_6(input_value):
  assert input_value % 6 == 0
```

fixtures can be auto-used, sometimes there can be fixtures that might need by all the test cases, in that case, we can use the “Auto use” fixture. eg: `@pytest.fixture(autouse=True)`

What are the scopes of pytest fixtures?

Some tests might need network activities like logging into a remote server or database, or if you are writing tests for machine learning models, you might have to train models which might be used by several test cases, in such cases running these fixture functions every time user requests can be time-consuming and computationally expensive.

In such situations we can define the scope of the fixtures eg: `@pytest.fixture(scope='...')`. The scope is simply when to teardown the object return by running a fixture function. There are 5 different scopes.

- **function:** when the scope is set to function, then the object will be teardown immediately after the requesting test function is terminated. When another test function calls that fixture it will rerun and create a new object. by default, the fixtures will have a function scope.
- **module:** when the first time a function calls the fixture it will create the object and save it for the usage of test functions in the same module. So if any of the test functions in the same module calls that fixture the cached object will be returned. The object will be torn down after all the tests in that module completes.
- **class:** the fixture will be executed once per class, it will be reused by all the test functions of that same test class. It will tear down after all the test functions from that class are completed.
- **package:** fixture will be executed on first request and cached until all the test functions in that class get executed, and will be torn down after that.
- **session:** this is the broad scope in pytest fixture, whenever we call `pytest` it is known as a session. So fixtures with session scope will be executed and cached on the first request and they will be reused until all the tests are completed.

Example

```
@pytest.fixture(scope="session")
def empty_account():
   print("\nempty_account fixture created with session scope")
   return Account(0)
```

```
@pytest.fixture(scope="module")
def account_20():
    print("\naccount_20 fixture created with module scope")
    return Account(20)


@pytest.fixture
def account_50():
    print("\naccount_50 fixture created with function scope")
    return Account(50)
```

Fixture example

Declare fixture. Pass it to the function against which you need fixture to be called.
Write yield keyword..anything written after that will be considered as tear down operation.

```
import pytest

@pytest.fixture()
def setUp():
    print("Launch browser")
    print("Login")
    print("Browse products")
    yield
    print("Logoff")
    print("Close browser")

def testAddItemtoCart(setUp):
    print("Add Item Successful")

def testRemoveItemFromCart(setUp):
    print("Remove Item Successful")
```

```
Login
Browse products
Add Item Successful
.Logoff
Close browser
Launch browser
Login
Browse products
Remove Item Successful
.Logoff
Close browser
```

Instead of copy/pastimg fixture in many files, use conftest.py and place common stuff there. i.e. place fixture there.
to further improve the situation..e.g. if you dont want to pass setup as an argument to testcases..: use autouse.

@pytest.fixture(autouse=True)
Session scope
@pytest.fixture(scope="session",autouse=True)
Since its for session, therefore setup will be called first but tear down is not called after add item testcase. Instead, it is called at the end of session (once the two test cases are finished operating)

```
test_cart.py::test_add_item_to_cart Launch browser
Login
Browse products
PASSED

test_cart.py::test_remove_item_from_cart PASSED
Logoff
Close browser
```

If Function scope

```
LearnPy/test_cart.py::test_add_item_to_cart Launch browser
Login
Browse products
Add Item Successful
PASSEDLogoff
Close browser

LearnPy/test_cart.py::test_remove_item_from_cart Launch browser
Login
Browse products
Remove Item Successful
PASSEDLogoff
Close browser
```

Parametrization In pytest
Parametrization is used to combine the multiple test cases into one test case. With parameterized testing, we can test functions and classes with different multiple sets of arguments.

In parametrize, we use `@pytest.mark.parametrize()` to perform parameterization in the Python code.

Example 1:
In this example, we are calculating the square of a number using the parametrization.

Create two files `parametrize/mathlib.py` and `parametrize/test_mathlib.py`

In `parametrize/mathlib.py` insert the following code that will return the square of a number.

```
def cal_square(num):
    return num * num
```

```

Save the file and open the second file` parametrize/test_mathlib.py`

In the test files, we write the test cases to test the Python code. Let’s use the Python test cases to test the code.

**Insert the following:**
```

```
import mathlib


# Test case 1
def test_cal_square_1( ):
    result = mathlib.cal_square(5)
    assert == 25



# Test case 2
def test_cal_square_2( ):
    result = mathlib.cal_square(6)
    assert == 36



# Test case 3
def test_cal_square_3( ):
    result = mathlib.cal_square(7)
    assert == 49



# Test case 4
def test_cal_square_4( ):
    result = mathlib.cal_square(8)
```

```
    assert == 64
```

There will be a number of test cases to test the code which is quite weird. The code for the test cases is the same except for the input. To get rid of such things, we will perform parameterization.

Replace the above test cases with the below:

```
import pytest
import mathlib


@pytest.mark.parametrize("test_input", "expected_output", [ (5, 25), (6, 36), (7, 49) ] )
    def test_cal_square(test_input, expected_output):
        result = mathlib.cal_square(test_input)
        assert result == expected_output
```

Skip and Xfail tests in Pytest.

Now, consider the below situations −

- A test is not relevant for some time due to some reasons.
- A new feature is being implemented and we already added a test for that feature.

In these situations, we have the option to xfail the test or skip the tests.

Pytest will execute the xfailed test, but it will not be considered as part failed or passed tests. Details of these tests will not be printed even if the test fails (remember pytest usually prints the failed test details). We can xfail tests using the following marker −

```
@pytest.mark.xfail
```

Skipping a test means that the test will not be executed. We can skip tests using the following marker −

```
@pytest.mark.skip
```

Later, when the test becomes relevant we can remove the markers.

Pytest - Stop Test Suite after N Test Failures

In a real scenario, once a new version of the code is ready to deploy, it is first deployed into pre-prod/ staging environment. Then a test suite runs on it.

The code is qualified for deploying to production only if the test suite passes. If there is test failure, whether it is one or many, the code is not production ready.

Therefore, what if we want to stop the execution of test suite soon after n number of test fails. This can be done in pytest using maxfail.

The syntax to stop the execution of test suite soon after n number of test fails is as follows −

```
pytest --maxfail = <num>
```

Create a file test_failure.py with the following code.

```
import pytest
import math

def test_sqrt_failure():
  num = 25
  assert math.sqrt(num) == 6

def test_square_failure():
  num = 7
  assert 7*7 == 40

def test_equality_failure():
  assert 10 == 11
```

All the 3 tests will fail on executing this test file. Here, we are going to stop the execution of the test after one failure itself by −

```
pytest test_failure.py -v --maxfail 1
```

Run tests in parallel

By default, pytest runs tests in sequential order. In a real scenario, a test suite will have a number of test files and each file will have a bunch of tests. This will lead to a large execution time. To overcome this, pytest provides us with an option to run tests in parallel.

For this, we need to first install the pytest-xdist plugin.

Install pytest-xdist by running the following command −

```
pip install pytest-xdist
```

Now, we can run tests by using the syntax **pytest -n <num>**

```
pytest -n 3
```

-n <num> runs the tests by using multiple workers, here it is 3.